MAGNIT TECH∞

# dbt
# трансформации как код

*ELT, слои, материализации,*
*тесты, документация, макросы*

*Курс Data Engineering | MAGNIT TECH x НИУ ВШЭ*

# План лекции

- Что такое ELT и где в стеке данных dbt?
- Как устроен dbt-проект: staging, intermediate, marts?
- Что такое модель, ref() и source()?
- Чем view отличается от table и incremental?
- Как проверять данные тестами?
- Откуда берётся документация и граф lineage?
- Зачем нужны Jinja и макросы?

MAGNIT TECH

# Философия ELT
# и место dbt в стеке

# Проблема: 30 запросов — 30 версий выручки

- **запросы разбросаны** — в BI, в папках, у разных людей;
- **логика повторяется** — выручку каждый считает по-своему;
- **проверок нет** — дубли заказов никто не замечает.

**Что даёт dbt**

Набор SELECT-ов превращается в проект: зависимости, тесты, документация и версии в Git.

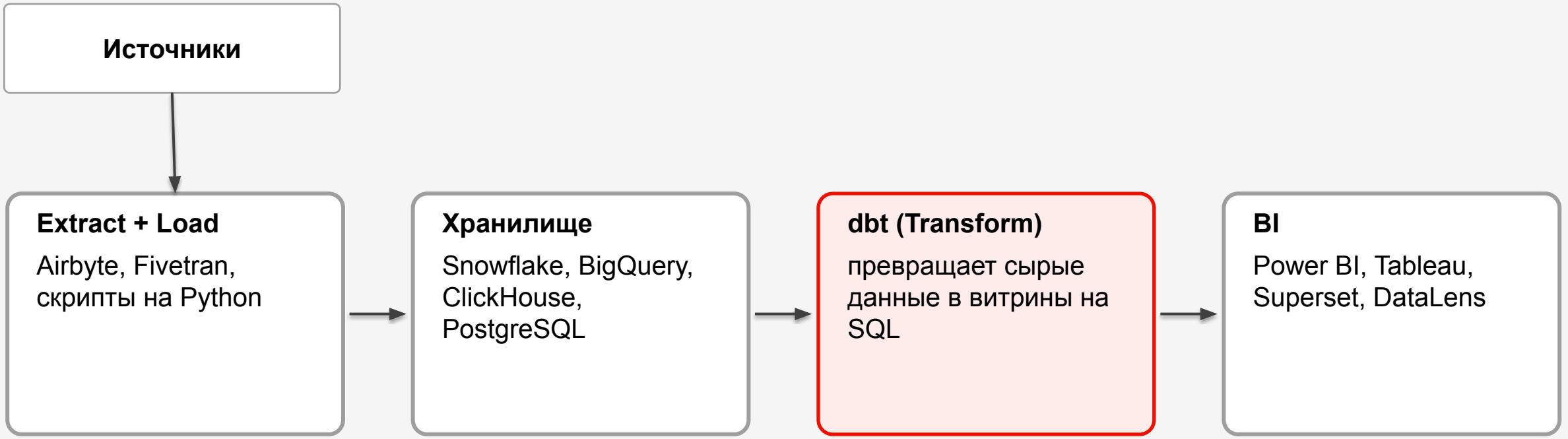
*Мы пишем только SELECT. dbt сам решает, в каком порядке запускать запросы и как сохранить результат в базу.*

# ETL или ELT

| | ETL — как было | ELT — как стало |
|---|---|---|
| **Порядок шагов** | Extract → Transform → Load | Extract → Load → Transform |
| **Где преобразуют** | на отдельном сервере, до хранилища | внутри хранилища, обычным SQL |
| **Сырые данные** | не сохраняются: ошиблись — тянем заново | лежат в хранилище: ошиблись — пересчитали |
| **Кто меняет логику** | инженеры в отдельном инструменте | аналитики и инженеры на SQL |

✅ *ELT стал возможен с облачными хранилищами — Snowflake, BigQuery, Redshift: хранить данные дёшево, а считают они очень быстро.*

# Современный стек: dbt — это буква T



MAGNIT TECH

# Структура проекта:
# staging, intermediate, marts

# Модель — это один SELECT

Модель — файл `.sql` с одним SELECT. CREATE TABLE и INSERT допишет dbt.

Имя файла станет именем таблицы в базе.

Вместо имён таблиц пишем `ref()` и `source()` — dbt подставит нужную схему сам.

```sql
-- models/marts/fct_orders.sql
select * from {{ ref('stg_shop__orders') }}
```

```sql
-- что уйдёт в базу после компиляции
select * from analytics.stg_shop__orders
```

*source() — ссылка на сырую таблицу, ref() — на другую модель. Из этих ссылок dbt строит граф зависимостей (DAG).*

# Структура папок проекта

Проект dbt — обычная папка с файлами. Её хранят в Git, как код любого приложения.

```
my_shop/
├── dbt_project.yml
├── models/
│   ├── staging/shop/
│   │   ├── _shop__sources.yml
│   │   ├── stg_shop__orders.sql
│   │   ├── stg_shop__payments.sql
│   │   └── stg_shop__customers.sql
│   ├── intermediate/
│   │   └── int_orders_with_payments.sql
│   └── marts/
│       ├── _marts.yml
│       ├── fct_orders.sql
│       └── dim_customers.sql
├── tests/
├── macros/
└── seeds/
```

***dbt_project.yml*** *— настройки проекта*

***models/*** *— модели, разложенные по слоям*

***_*.yml*** *— описания и тесты моделей*

***tests/*** *— singular-тесты*

***macros/*** *— макросы*

***seeds/*** *— небольшие CSV-справочники*

MAGNIT TECH∞

# Данные текут в одну сторону

**1**

**Источники**

сырые таблицы, их загрузил EL-инструмент

→

**2**

**Staging**

stg_: переименовать, привести типы

→

**3**

**Intermediate**

int_: JOIN-ы, группировки

→

**4**

**Marts**

fct_, dim_: готовые витрины

→

**5**

**BI**

дашборды читают только марты

*Источники читает только staging. Переименовали колонку в источнике — правим один файл, а не тридцать отчётов.*

# Три слоя: кто что делает

fct_ — факты (события: заказы, платежи), dim_ — измерения (справочники: клиенты, товары).

| Слой | Что делает | Чего там нет |
|---|---|---|
| **Staging · stg_** | один-в-один с источником: имена, типы, копейки → рубли | JOIN-ов и агрегаций |
| **Intermediate · int_** | промежуточные шаги: JOIN-ы, группировки, сложная логика | обращений к source() |
| **Marts · fct_, dim_** | готовые витрины для аналитиков и BI | технического мусора |



*Зачем три файла вместо одного запроса? Каждую часть можно переиспользовать и тестировать отдельно.*

# Один заказ через три слоя

**STAGING**

Чистим источник: понятные имена, типы, рубли вместо копеек.

```
select
  id as order_id,
  user_id as customer_id,
  amount_kopecks / 100.0
    as amount_rub
from {{ source('shop',
                'orders') }}
```

**INTERMEDIATE**

Соединяем заказы с оплатами и суммируем.

```
select
  o.order_id,
  o.customer_id,
  sum(p.amount_rub)
    as paid_rub
from
  {{ ref('stg_shop__orders') }} o
left join
  {{ ref('stg_shop__payments') }} p
  on o.order_id = p.order_id
group by 1, 2
```

**MARTS**

Готовая витрина: одна строка — один заказ.

```
select
  order_id,
  customer_id,
  paid_rub,
  paid_rub > 0 as is_paid
from {{ ref(
  'int_orders_with_payments') }}
```

MAGNIT TECH

MAGNIT TECH

# Материализации:
# view, table, incremental

# Во что превратится мой SELECT

## view · по умолчанию

Сохранённый запрос: create view … as select …

Собирается мгновенно, данные всегда свежие, но читается медленно.

Для staging.

## table

Таблица пересоздаётся целиком при каждом dbt run.

Читается быстро, но каждый запуск считает всё заново.

Для мартов под дашборды.

## incremental

Первый раз создаёт таблицу, дальше добавляет только новые строки.

Быстро на больших объёмах, но сложнее писать.

Для логов, кликов, транзакций.

*Ещё есть ephemeral (подзапрос, в базе не создаётся) и snapshots (история изменений строк) — тема отдельного занятия.*

# Как задать материализацию

### Для папки — dbt_project.yml

```yaml
models:
  my_shop:
    staging:
      +materialized: view
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
```

### Для одной модели — config() в начале файла

```sql
{{ config(materialized='table') }}

select
  order_id,
  paid_rub
from {{ ref('int_orders_with_payments') }}
```

*Настройка в файле модели перебивает настройку папки.*

# Incremental: только новые строки

Таблицу кликов на миллиард строк пересчитывать каждую ночь целиком долго и дорого.

**is_incremental()** — правда, если таблица уже есть. При первом запуске фильтра нет.

**{{ this }}** — сама эта таблица в базе.

**unique_key** — строка с тем же id обновится, а не задвоится.

```
{{ config(
    materialized='incremental',
    unique_key='event_id'
) }}

select event_id, user_id,
       event_type, event_time
from {{ ref('stg_app__events') }}

{% if is_incremental() %}
  where event_time >
    (select max(event_time) from {{ this }})
{% endif %}
```

*Логика поменялась? Пересобрать с нуля:*

*dbt run --full-refresh*

**1 млрд**

строк в таблице кликов

**только новые**

строки читает каждый следующий запуск

MAGNIT TECH

MAGNIT TECH∞

# Тесты:
# generic и singular

# Тест — запрос, который ищет плохие строки

Самая страшная ошибка в аналитике — тихая: дашборд рисуется, а выручка завышена вдвое из-за дублей заказов.

Тест в dbt проверяет не код, а сами данные в таблицах.

```sql
select order_id, count(*)
from fct_orders
group by order_id
having count(*) > 1
```

```
0 строк        → PASS
хотя бы 1 строка → FAIL
```

**0 строк**

вернул запрос — тест пройден

*Так выглядит под капотом тест unique для order_id. Сами такие запросы мы не пишем — dbt генерирует их из YAML.*

# Четыре встроенных generic-теста

**1** **unique**
в колонке нет повторов: order_id не дублируется

**2** **not_null**
нет пустых значений: у каждого заказа есть customer_id

**3** **accepted_values**
значения только из списка: new, paid, shipped, cancelled

**4** **relationships**
каждый customer_id из заказов есть в dim_customers

*Ещё больше готовых тестов — в пакетах dbt_utils и dbt_expectations: например, «значение от 0 до 100».*

**4**
generic-теста есть в dbt из коробки

**1 строка**
YAML подключает тест к колонке

**любые**
модели и колонки — один тест на всех

# Generic-тесты пишутся в YAML

Файл models/marts/_marts.yml лежит рядом с моделями.

```yaml
models:
  - name: fct_orders
    columns:
      - name: order_id
        data_tests:
          - unique
          - not_null
      - name: customer_id
        data_tests:
          - relationships:
              to: ref('dim_customers')
              field: customer_id
      - name: order_status
        data_tests:
          - accepted_values:
              values: ['new', 'paid',
                       'shipped', 'cancelled']
```

*Каждую строчку dbt превращает в SQL-запрос вроде того, что на прошлом слайде.*

*В старых примерах вместо data_tests: написано tests: — это то же самое, старое имя.*

# Singular-тест: своя проверка

Иногда правило уникально для вашего бизнеса: «сумма оплаты не бывает отрицательной».

Пишем обычный SQL-файл в папке tests/, который выбирает нарушителей.

```sql
-- tests/assert_no_negative_payments.sql
select *
from {{ ref('fct_orders') }}
where paid_rub < 0
```

```
dbt test                      -- все тесты
dbt test --select fct_orders -- одной модели
dbt build                     -- собрать + проверить
```

*dbt build ставят в расписание: сломанные данные из staging не дойдут до дашборда.*

# Generic или singular

Оба вида — SQL-запросы, которые ищут плохие строки. Разница в том, где и для чего их пишут.

| | Generic | Singular |
|---|---|---|
| **Где пишется** | YAML-файл рядом с моделью | отдельный .sql в папке tests/ |
| **Сколько раз применяется** | к любому числу колонок и моделей | один раз, для одной ситуации |
| **Пример** | «ключ уникален» | «оплата не бывает отрицательной» |

MAGNIT TECH

# Документация
# рядом с кодом

# Описания в тех же YAML

Документация в Confluence устаревает в день написания. В dbt описание лежит рядом с кодом и меняется в том же коммите.

```yaml
models:
  - name: fct_orders
    description: "Одна строка = один заказ"
    columns:
      - name: order_id
        description: "Номер заказа"
      - name: is_paid
        description: "Есть хотя бы
          одна оплата"
```

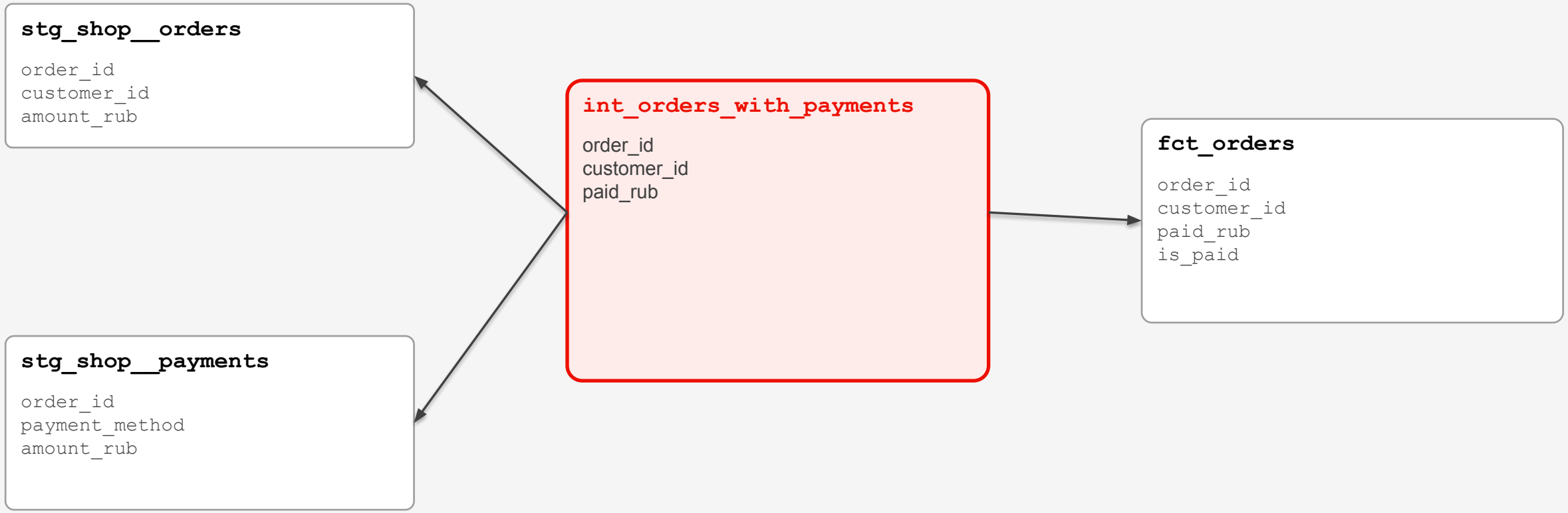
*description — у модели и у каждой колонки.*

```
-- собрать документацию
dbt docs generate

-- открыть её в браузере как сайт
dbt docs serve
```

*Типы колонок и размеры таблиц dbt подтянет из базы сам.*

# Граф lineage строится сам




*dbt рисует граф из ref() и source(). «Откуда эта цифра?» и «что сломается, если удалить колонку?» — ответ на картинке.*

MAGNIT TECH

# Макросы и Jinja: программируемый SQL



# Jinja: три вида скобок

Перед запуском dbt компилирует модель: выполняет Jinja и отправляет в базу чистый SQL. База про Jinja ничего не знает.

| Синтаксис | Что делает | Пример |
|---|---|---|
| `{{ ... }}` | выводит значение в текст SQL | `{{ ref('stg_orders') }}` |
| `{% ... %}` | команда: условие, цикл, переменная | `{% if ... %} … {% endif %}` |
| `{# ... #}` | комментарий, в SQL не попадёт | `{# TODO: убрать тестовые заказы #}` |



*Скомпилированный SQL можно посмотреть: dbt compile → папка target/compiled/.*

# Цикл вместо копирования строк

## Пишем мы

```
{% set methods = ['card', 'sbp', 'cash']%}
select
  order_id,
  {% for m in methods %}
  sum(case when payment_method = '{{ m }}'
      then amount_rub else 0 end)
    as {{ m }}_rub
  {%- if not loop.last %},{% endif %}
  {% endfor %}
from {{ ref('stg_shop__payments') }}
group by order_id
```

## Уходит в базу

```
select
  order_id,
  sum(case when payment_method = 'card'
      then amount_rub else 0 end) as card_rub,
  sum(case when payment_method = 'sbp'
      then amount_rub else 0 end) as sbp_rub,
  sum(case when payment_method = 'cash'
      then amount_rub else 0 end) as cash_rub
from analytics.stg_shop__payments
group by order_id
```

*loop.last — «последний шаг цикла»: нужен, чтобы после последней колонки не было лишней запятой.*

# Макрос — функция для SQL

```
-- macros/kopecks_to_rub.sql
{% macro kopecks_to_rub(column_name,
                        precision=2) %}
  round({{ column_name }} / 100.0,
        {{ precision }})
{% endmacro %}
```

```
-- используем в модели
select
  id as order_id,
  {{ kopecks_to_rub('amount_kopecks')  }}
    as amount_rub
from {{ source('shop', 'orders')  }}
```



*Поменялось правило округления — правим один файл, а не двадцать моделей. Принцип DRY: don’t repeat yourself.*

# Пакеты: готовые макросы

## generate_surrogate_key

Уникальный ключ из нескольких колонок.

Пример:

```
dbt_utils.generate_surrogate_key
(['order_id', 'line_no'])
```

## star

Все колонки таблицы, кроме перечисленных.

Пример:

```
dbt_utils.star(ref('stg_orders')
, except=['_loaded_at'])
```

## date_spine

Таблица-календарь: одна строка на каждый день.

Подключение:

файл packages.yml + команда dbt deps

# Итоги

- dbt — буква T в ELT: преобразует данные внутри хранилища SQL-запросами.
- Модель — один SELECT; ref() и source() строят граф зависимостей.
- Слои: staging чистит, intermediate собирает, marts отдаёт бизнесу.
- Материализация — одна строка: view, table или incremental.
- Тест — запрос, который ищет плохие строки: generic в YAML, singular в tests/.
- Документация и lineage собираются из YAML, повторы уходят в макросы.

MAGNIT TECH

# Спасибо за занятие!